

Modularity Makes Dexterous Hands Learn Faster, Not Better

Anonymous authors (double-anonymous review)

Abstract—Modular motion models such as MotionBricks build motion from reusable parts, and the idea is attractive for dexterous hands: a hand that learns in modules should learn faster and reach further. We test both promises, with every prediction registered before the runs. Used as the action space of a dexterous policy, a MotionBricks-style keyframe interface learns held-out skill sequences far better early in training (0.59 against 0.22 success) but a continuous latent catches up by the end (0.49 against 0.44). Separately learned skills do no better than one flat policy trained on the same demonstrations (45 against 61 of 400 test episodes), and a hand-off defined too loosely let one skill fling the ball out of the next one’s reach. The one gain, choosing the best policy for each stage (97 against 61), needs no separate training. Modularity makes dexterous hands learn faster, not better, and four conditions explain when it would pay. To choose which motion data to test on, we also built an instrument for whether hand-motion priors generalize to unseen combinations: over nine axes of three datasets, difficulty appears only where the two factors interact, a statistic computed on raw poses in minutes.

Index Terms: *Dexterous Manipulation, Reinforcement Learning, Learning from Demonstration, Multifingered Hands, Deep Learning Methods.*

I. INTRODUCTION

MotionBricks [1] generates character motion by composing reusable parts: poses joined by a learned in-betweeners, and smart primitives that act. Modularity of this kind promises two things. A learner that works in modules should learn faster, because each module is simpler than the whole task; and it should reach further, because modules recombine into behaviour never seen whole. Dexterous hands are a natural place to want both, since their tasks are long and their data are scarce.

We test both promises on dexterous hands, pre-registered. The answer is the title: modularity makes dexterous hands learn faster, not better. A keyframe interface in MotionBricks’ sense learns held-out skill sequences almost three times as well at 1.5 M frames, and a continuous latent catches up by 5 M. Skills trained separately do no better than one flat policy trained on the same demonstrations, and chaining them only works once each hand-off is defined strictly. The only gain at the skill level came from choosing the best policy for each stage, and that works just as well with flat-trained policies. Four conditions explain why: the solution must lie inside the modules, hand-offs must be clean, there must be something to amortise, and a flat learner must be unable to get the same benefit. MotionBricks meets all four in animation; the dexterous tasks here fail at least one each.

Testing recombination needs motion data where recombination is actually hard, and measuring that is expensive. A composition axis pairs two labelled factors, such as object category and intent; the measurement trains one prior that saw a held-out pairing and one that did not, over many

seeds, at hours of GPU per axis. Most axes, we find, carry no difficulty at all. So we also built a way to find the ones that do. Our contributions:

- 1) **Modularity for dexterous control**, tested at both of MotionBricks’ layers with every prediction registered: faster learning at the motion level, no higher final success, and no gain from training skills separately (Section IV).
- 2) **Four conditions** under which modularity pays, which account for both layers and for when MotionBricks succeeds (Section V).
- 3) **An instrument and a cheap screen** for compositional difficulty in hand motion: a paired split, five gates, a zero-truth control, and an interaction statistic on raw poses that says which axes to skip (Section VI).

II. RELATED WORK

Priors as action spaces. LAMP [2] uses a continuous latent prior as the action space for residual RL, and DexSynRefine [3] grounds motion primitives with residual RL. Synergy, latent and dynamical-system action spaces mostly report faster learning [4]–[8], and PPO on raw actions can still end higher [8]; higher final success is reported where the joint-space baseline had not converged [9], [10]. None tests generalization to unseen combinations.

Modular motion. Modularity appears as authored primitives over a latent backbone [1], switching dynamical systems [11], [12] and mode-adaptive gating [13]. Skill chaining trains each skill to start where the previous one ends [14] and regularises terminal states so the next skill can take over [15]; on dexterous hands it works when the hand-offs are engineered [16]. These modular gains come with such extra machinery, so they do not separate modular training from what is added to it. Most of the benefit of hierarchy in RL has been traced to exploration [17], and its efficiency to sub-problems that repeat [18].

Policy selection. Imitation-learned policies vary widely across seeds and checkpoints, and which one is chosen matters as much as how it was trained [19], [20]. That work selects whole policies. We select per stage, and find that this, not separate training, is where the skill-level gain of modularity lies.

Compositional generalization. The only held-out pairing for hand motion we know of is TACO’s novel-triplet test set [21]. It is a single unpaired split, so its gap mixes composition with any other difference between the two test sets. We pair the arms on identical test data and ask which axes carry difficulty at all.



Fig. 1. A 90%-flexion reference pose (left) and its projection from OakInk-Image, GRAB and DexYCB (left to right) with conventions inferred per joint. Thumb red, index orange, middle yellow, ring green, little blue.

III. HAND-MOTION PRIORS

Hand representation. Motion is 27 anatomical degrees of freedom in native units: four per finger (MCP flexion and abduction, PIP, DIP), five for the thumb and six for the wrist. Normalization into $[-1, 1]$ comes from the joint-limit box, never from dataset statistics, so a prior trained on one dataset stays meaningful on another.

Datasets provide MANO [22] parameters. Projecting them onto an anatomical model needs three conventions per joint: which Euler axis carries flexion, its sign, and which of the remaining two carries abduction. We infer all three per joint from the data instead of assuming them. Assumed conventions fail in both directions: a fixed abduction axis put 100% of GRAB [23] and DexYCB [24] frames outside the thumb’s 20° abduction range, and pooling PIP variance across fingers pinned OakInk2’s [25] index PIP at its limit in every frame. Inferring per joint halves OakInk2’s projection residual and takes its PIP joints from 37.3% to 99.9% inside their range. Fig. 1 shows the three datasets agreeing on a reference pose.

The priors. Our continuous prior encodes a 32-frame window of the 27 DOF into a 12-D Gaussian latent per frame and decodes it under a bounded output, so joint limits hold by construction. Bidirectional two-layer GRUs of 256 units encode and unidirectional ones decode, 2.28 M parameters in all. It is trained as a β -VAE [26]: mean-squared error on pose, a first-difference term for smoothness, and β times a KL term with a free-bits floor of 0.02 nats per dimension, β annealed over the first epochs. The latent does not collapse (mean active-unit fraction 0.996 over 140 trained models). A modular bank of matched size ($K = 12$ primitives, 2.30 M parameters) was trained beside it; Section IV explains why we compare bank and latent as action spaces rather than by reconstruction.

IV. DOES MODULARITY HELP CONTROL?

Reconstruction cannot compare a modular bank with a continuous latent fairly, because they differ in information rate: once the latent’s rate is raised to match, the interpolable control, which has nothing to compose, reproduces 90% of the bank’s reconstruction advantage (Section VI). A policy can compare them. We used each prior as the action space of PPO [27] on tasks that must be solved, and asked whether its structure helps. Every design choice, prediction and deviation was registered before the runs it governed, and the registration is part of the code release.

Interfaces. All interfaces add the same $\pm 15^\circ$ per-frame residual on top of the prior’s output. (a) The $K = 16$ clip

TABLE I
EACH PRIOR AS A POLICY’S ACTION SPACE: SUCCESS ON SKILL SEQUENCES NEVER SEEN IN TRAINING (STAGED TASK) AND ON TWO ADROIT TASKS.

interface	staged task, held-out		Adroit, 3 M	
	1.5 M	5 M	hammer	pen
plain PPO (no prior)	—	0.22	1.00	0.49
continuous latent	0.22	0.49	—	—
keyframe, in-betweeners	0.59	0.44	1.00	0.12
keyframe, straight-line	0.00	—	—	—
$K = 16$ clip bank	0.08	—	—	—
8-pose library	—	—	0.00	0.10
RL seeds	20 (10)	10	1	1

bank: the policy picks a soft mixture of primitives and a style every 16 frames. (b) The continuous latent: the policy emits the 12-D latent every frame. (c) The keyframe interface, in MotionBricks’ sense [1]: the policy emits a target pose every 16 frames and a GRU in-betweeners, trained on the same human motion, fills the frames between. (d) The same keyframes reached by straight-line interpolation, with no learned prior. (e) On Adroit, a library of eight poses clustered from OakInk. (f) Plain PPO on joint targets.

Motion-level modularity. On the Shadow Hand E3M5 [28] in MuJoCo [29], a 2.5 cm, 0.20 kg cube rests on the palm under a lateral tilt of 1.5 g. An episode is a sequence of skills of forty frames each: hold under an x tilt, hold under a y tilt, release. Policies train on the four two-skill sequences and are scored on the two three-skill sequences they never saw, over 100 episodes with the deterministic policy.

At 1.5 M frames the keyframe interface won clearly: 0.59 against the latent’s 0.22 on held-out sequences (difference $+0.38$, t -test $p < 0.001$, 17 of 20 seeds, sign test $p = 0.003$, as declared). It composed learned skills without loss, and uniformly so at three, four and five skills (0.59, 0.51, 0.48 against 0.22, 0.22, 0.15). The same keyframes reached by a straight line never learned to release (0 of 10 seeds), so the learned in-betweeners, not the keyframes alone, carries the gain.

The gain is one of speed. At 5 M frames the latent caught up, 0.49 against 0.44 ($p = 0.58$, Table I): every interface then holds perfectly (0.96 to 1.00 per skill) and releases about half the time. The keyframe interface learned the holds by 2.5 M frames, where the latent needed 5 M. Either learned prior beats plain PPO on this task (0.44 to 0.49 against 0.22). The bank was the worst action space throughout, and on plain retention no better than a randomly initialised decoder of the same shape (0.17 against 0.24). The reason is controllability: from any start pose its primitives move the fingers half as far in 30 frames as the latent does (5.9° against 12.2°).

On Adroit [30] (Gymnasium-Robotics 1.4.2, dense rewards, 200-step episodes) the pattern was the same: where PPO learned the task (hammer 1.00, pen 0.49), the priors were slower or worse, and where it did not (door, relocate), they did not unlock it.

Skill-level modularity. MotionBricks’ other layer composes smart primitives, units that act rather than poses. We tested it on Adroit’s relocate, extended into a sparse multi-

TABLE II

SKILL-LEVEL MODULARITY ON STAGED RELOCATE (STRICT STAGES: 25 CONSECUTIVE STEPS)

setting	success
grasp, behaviour cloning	0.70
carry, behaviour cloning, after a real grasp	0.18
carry, PPO from scratch, dense reward, 5M steps	0.00
flat PPO on the whole task	0.00
modular vs flat, 2 stages (of 200 episodes)	38 vs 8
modular vs flat, 4 stages (of 200 episodes)	5 vs 2

stage task: grasp the ball from the table, then carry it to a random target in the air, once or several times. The data are the 25 human demonstrations of D4RL [31], each containing one grasp and one carry. The modular arm trains one policy per skill and chains them with a fixed sequencer; the flat arm trains one policy on the whole task with the stage index in its input. Table II collects the results.

Two things decided the outcome. First, hand-offs have to be defined strictly. With a stage that counts as done at a single step, carry policies flung the ball through the target at a median of 3.7 m/s, a failure of the kind skill chaining is known to invite [15]. From those end states the ball dropped within about 18 steps in 98 of 100 episodes, and neither continuing the carry nor freezing every joint kept it (0 of 40 each). Every stage therefore has to hold for 25 consecutive steps, as Adroit’s own success criterion does, and all results reported here use that criterion. Second, the skills came only from the demonstrations: PPO from scratch never completed a carry in 5 M steps, even with shaped reward, and flat PPO on the whole task scored 0.

With the demonstrations shared, separately and jointly cloned policies were not distinguishable on average (product of the two stage rates 0.028 against 0.032, five seeds each). The declared test at four stages was at the floor (5 against 2 of 200, Fisher $p = 0.45$). At two stages the modular arm won, 38 against 8 of 200 ($p < 10^{-5}$), which we traced, after the fact, to picking the best grasp and the best carry separately among five seeds each.

Training or selection? We then registered a test that separates the two, on fresh seeds and fresh test episodes, at a matched budget of 20 trainings per side. Arm A is the best of 20 flat policies, used for every stage. Arm B takes the same 20 flat policies and uses the best grasper for the grasp and the best carrier for every carry. Arm C trains 10 grasp skills and 10 carry skills separately and chains the best of each. All choices use 200 validation episodes; the test is 400 episodes never used for any choice (Table III).

Our declared prediction, that separately trained skills beat the best flat policy, failed: arm C scored 45 against 61 at two stages, in the wrong direction. Separate training was worse, not better: carry skills cloned from carry segments alone reached 0.14 on validation against 0.27 for the carry stage of flat policies ($p < 10^{-5}$); the flat policies had learned from the whole demonstrations. Per-stage selection, the other

TABLE III

TRAINING OR SELECTION: ALL-STAGE SUCCESS ON 400 FRESH TEST EPISODES, PRE-REGISTERED. FISHER’S EXACT TEST AGAINST ARM A.

arm	2 stages	3 stages
A: best flat policy, every stage	61	12
B: best flat policy per stage	97 ($p = 0.002$)	47 ($p < 10^{-5}$)
C: separately trained skills	45 ($p = 0.12$)	6 ($p = 0.23$)

TABLE IV

THE FOUR CONDITIONS FOR MODULARITY TO PAY, AGAINST THE TWO LAYERS TESTED

condition	motion level	skill level
(i) solution inside	no: 83° grip needed, 73–76° in our datasets	yes: skills learned on the task
(ii) clean hand-offs	yes: poses only	only when enforced: a loose stage let carry fling the ball
(iii) something to amortise	partly: faster exploration, one task	yes: flat PPO scores 0
(iv) flat cannot match	no: caught up at 5 M	no: flat policies selected per stage do best

half of the prediction, held: arm B beat arm A by 9 points at two stages ($p = 0.002$) and quadrupled it at three (47 against 12). The skill-level gain is therefore not modular training. It is choosing each stage’s policy on its own, and a flat learner gets it by being split by stage at deployment.

V. DISCUSSION: WHEN DOES MODULARITY PAY?

A chain of modules succeeds with probability $\prod_i P(\text{module } i \mid \text{end states of module } i-1)$, so every benefit of modularity is paid for at the hand-offs. Modularity pays when four conditions hold: (i) the solution lies inside what the modules can express; (ii) hand-offs are clean; (iii) there is something to amortise, such as a horizon a flat learner cannot cross or combinations it has not seen; (iv) a flat learner given the same data cannot get the same benefit. Table IV shows that each layer we tested fails at least one.

A flat policy that knows which stage it is in can already represent the modular solution. So in RL, modular versus flat is a comparison of training procedures (separate objectives, start states and data), not of architectures, consistent with the finding that most of the benefit of hierarchy is better exploration [17]. Work on synergy and latent action spaces reports the same trade as our motion-level result, faster early learning with an equal or lower final success [4]–[7]; higher final success has been reported where the joint-space baseline had not converged [9], [10], whereas ours reached 1.00 on hammer. Skill chaining on dexterous hands succeeds when the hand-offs are engineered [16], which is condition (ii).

Modularity does offer one benefit that survived: each stage can be validated and selected on its own. With N trained policies, per-stage selection picks the best of each stage, while whole-policy selection can only pick the best product. The product of the per-stage best is never below the best whole policy, and it is strictly above it whenever the best grasp and the best carry come from different seeds. Table III

TABLE V
THE FIVE GATES, AND WHAT EACH CAUGHT

Gate	What it caught
primitive collapse	<code>primitives_used</code> = 1.0, one primitive active in every frame, a modular model that is monolithic in disguise
informed coverage	held compositions absent from the informed set, which pins the penalty at zero by construction
composition leak	targets whose exact fine composition sat in the informed training set, against none for naive
DOF health	<code>thumb_mcp_abd</code> pinned at its limit in every GRAB and DexYCB frame under an assumed abduction axis
split balance	guard only; the risk that an unpaired design measures set difficulty rather than compositional difficulty

shows that this benefit belongs to modular *deployment*, not to modular training: it was largest with policies trained flat. We have not found it reported before; work on policy selection picks whole policies [19], [20], and work on skill chaining credits the gain to training [15], [16]. MotionBricks meets all four conditions in animation: its goal is human-like motion and its data are human motion, it is kinematic so any pose can start the next brick, one library serves many applications, and it has 10^5 clips. Each dexterous layer we tested fails at least one.

VI. WHERE IS RECOMBINATION HARD IN HAND-MOTION DATA?

A modular prior is only worth testing on recombination where recombining is actually hard. This section gives the instrument we built to find where that is, and what it found.

Paired measurement. We score the same held-out trajectories T under two priors that differ only in whether their training data contained T ’s composition:

$$\text{penalty}(T) = \text{MSE}_{\text{naive}}(T) - \text{MSE}_{\text{informed}}(T).$$

A composition is one left $\times$ right cell, for example *mug* $\times$ *pour*. Each seed draws its own split, which holds out four to eight compositions; a composition is held out only if it spans several distinct fine labels (object ids or primitive chains), so the informed arm can see the composition without seeing the target itself. Both arms see the same number of training trajectories and are scored on the same test data, so trajectory difficulty and set size cancel, and the two arms’ evaluation noise is correlated. The difficulty of an axis is the penalty averaged over seeds.

An unpaired design reads set imbalance as difficulty: on OakInk-Image the variance of the unseen set differs from the seen set’s by -6.1% to $+10.7\%$ across five seeds. Pairing removes that by construction. Table V lists the five gates every sweep passes through; each of the first four caught a real defect in an earlier design, and the fifth guards against the imbalance above.

The screen. The penalty asks whether knowing the *pairing* helps beyond knowing each factor, which is an interaction.

On PCA-reduced poses we fit

$$\begin{aligned} \text{additive: } & \text{pose} \sim \text{left} + \text{right}, \\ \text{full: } & \text{pose} \sim \text{left} + \text{right} + \text{cell}, \end{aligned}$$

and take the interaction $R^2(\text{full}) - R^2(\text{additive})$ against a permutation null that shuffles labels over whole trajectories, since windows of one trajectory are near-duplicates. The *excess* is the interaction above the null mean, and z its distance in null standard deviations. The screen needs no trained model and runs in minutes on a CPU, against hours of GPU for a sweep.

A simpler statistic, the variance the composition label explains, does not work. GRAB (+0.0538 above its null) and OakInk-Image (+0.0547) are indistinguishable on it, yet GRAB’s axis carries no difficulty and OakInk-Image’s does. The interaction separates them, at $z = 3.9$ on GRAB against $z = 8.3$ on OakInk-Image’s category axis.

Setup.

We use OakInk-Image [32], OakInk2 [25], GRAB [23] and DexYCB [24], plus two synthetic controls: one with planted compositional difficulty and one purely interpolable, whose true penalty is zero. Composition axes come from each dataset’s own taxonomy: OakInk ships object categories, functional classes and affordances, GRAB object shapes and intents, and OakInk2 scenes, verbs and primitives. Sweeps train 256 trajectories per arm for 120 epochs on 32-frame windows, over 12 to 130 seeds per axis. The interpolable control still returns +2.6% of the naive error (20 seeds, sd 2.3 points), so the instrument has a bias, and we read every axis against this control with Welch’s test. The test against zero, which the pre-registration declared, is reported beside it.

Table VI and Fig. 2 give the nine axes whose sweeps are interpretable; six more were screened but could not be split without leakage or failed the coverage gate. The category axis carries the confirmatory claim; the affordance axis is exploratory, and the id-prefix axis is an older axis kept for comparison. Table VII lists the six axes that were screened but not reported. All six have negative excess, so the screen would have skipped them anyway, and none of them yields clean splits at the settings we used.

Interaction is necessary. Every axis that carries difficulty has positive interaction, 3.8 to 11.2 standard errors above the control (category $\times$ intent: +15.2%). The three swept axes without positive interaction sit at the control (+2.3% to +3.4%, $p = 0.69$ to 0.78).

The two forty-seed null axes are significant against zero ($p = 0.021$ and 0.015). Read against zero, the instrument’s own bias would have counted as difficulty; read against the control, it does not.

It is not sufficient. Two axes with positive interaction, GRAB’s shape $\times$ fine intent ($z = 3.9$, +0.7%) and OakInk2’s scene $\times$ primitive ($z = 2.9$, +2.3%), are also at the control. Informative factors are not enough either: OakInk2’s factors explain the most pose variance of any axis (0.188 and 0.243), and pairing them adds nothing measurable.

TABLE VI

THE INTERACTION SCREEN AGAINST THE MEASURED DIFFICULTY, SORTED BY EXCESS. DIFFICULTY IS THE PAIRED PENALTY OVER n SEEDS, ALSO AS A PERCENTAGE OF THE NAIVE ERROR; p IS FROM A TEST AGAINST ZERO AND FROM WELCH’S TEST AGAINST THE ZERO-TRUTH CONTROL.

dataset, axis	η^2 left	η^2 right	excess (z)	difficulty	% naive	p vs 0	p vs control	n
OakInk-Image, affordance $\times$ intent (exploratory)	0.054	0.045	+0.0309 (14.9)	+0.00873	+12.2	$< 10^{-12}$	$< 10^{-12}$	130
OakInk-Image, category $\times$ intent (confirmatory)	0.097	0.045	+0.0238 (8.3)	+0.01042	+15.2	$< 10^{-12}$	$< 10^{-12}$	69
OakInk-Image, id-prefix $\times$ intent (deprecated)	0.028	0.045	+0.0182 (12.9)	+0.00982	+14.3	$< 10^{-12}$	$< 10^{-12}$	70
GRAB, shape $\times$ fine intent	0.028	0.044	+0.0063 (3.9)	+0.00041	+0.7	0.494	0.23	40
OakInk-Image, class $\times$ intent	0.022	0.045	+0.0061 (4.1)	+0.01650	+17.9	0.0018	0.0029	12
OakInk2, scene $\times$ primitive (confound test)	0.056	0.188	+0.0040 (2.9)	+0.00039	+2.3	0.068	0.82	40
OakInk2, scene $\times$ verb (pre-registered)	0.056	0.175	−0.0001 (−0.1)	+0.00051	+3.0	0.015	0.78	40
GRAB, shape $\times$ intent class (pre-registered)	0.028	0.009	−0.0005 (−0.4)	+0.00103	+2.3	0.021	0.77	40
OakInk2, annotated transitions	0.188	0.243	−0.0120 (−2.6)	+0.00082	+3.4	0.115	0.69	12
synthetic, via-point (planted difficulty)	—	—	—	+0.00930	+9.1	$< 10^{-7}$	$< 10^{-6}$	18
synthetic, interpolable (control, true zero)	—	—	—	+0.00229	+2.6	< 0.0001	—	20

TABLE VII

SIX SCREENED AXES THAT COULD NOT BE SWEEPED CLEANLY

dataset, axis	excess (z)	reason
DexYCB, subject $\times$ shape	−0.0214 (−2.4)	informed arm misses held compositions
DexYCB, subject $\times$ object	−0.0899 (−8.9)	one fine label per composition
GRAB, object $\times$ fine intent	−0.0134 (−6.8)	one fine label per composition
GRAB, object $\times$ intent class	−0.0128 (−7.6)	coverage fails 19–33 of 40 splits
OakInk2, subject $\times$ primitive	−0.0333 (−4.1)	coverage fails 13 of 40 splits
OakInk2, subject $\times$ verb	−0.0360 (−4.7)	coverage fails 8 of 40 splits

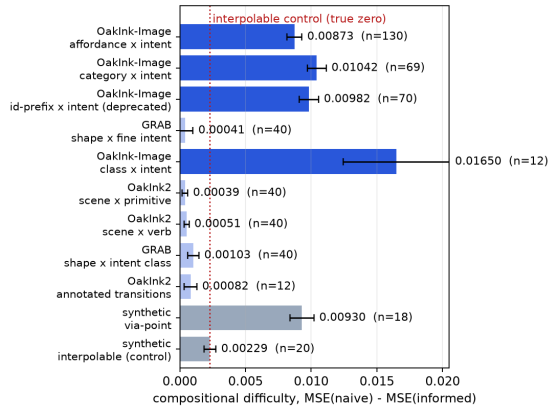


Fig. 2. Compositional difficulty per axis (mean and standard error over seeds). Faded bars are indistinguishable from the zero-truth control (dotted line).

It does not predict magnitude. Across the nine axes, excess and difficulty correlate at $r = 0.58$ ($p = 0.10$). The class axis was swept on the screen’s own recommendation, with its reading declared in advance: ranked fifth by excess, it was predicted to be small, and it returned the largest difficulty we measured (+17.9%). Use the screen to decide what to skip, and the sweep to measure.

Robustness. The confirmatory estimate does not depend on marginal seeds: dropping the five category seeds whose

informed arm missed one held composition leaves the axis at +0.01042 over 64 seeds, +15.2% of the naive error, unchanged to five decimals. Both halves of the category and affordance sweeps sit at least 6.1 standard errors above the control on their own.

VII. LIMITATIONS

Every axis with difficulty comes from one dataset, so interaction and dataset are not yet separated. The reading against the control was chosen after the declared test against zero had been run. The control experiments use one simulated hand, and the Adroit comparisons use one seed per task.

VIII. CONCLUSION

Modularity makes dexterous hands learn faster, not better. A MotionBricks-style keyframe interface gives an early lead that a continuous latent closes, and separately trained skills do no better than one flat policy. Where modularity helped at the skill level, it was by choosing the best policy for each stage, and that works with policies trained flat. Before building a modular policy, check the four conditions, starting with the hand-offs. Before spending GPU on a composition axis, compute the interaction statistic, and read every penalty against a zero-truth control.

ACKNOWLEDGMENT

Generative AI (Claude, Anthropic) was used under the authors’ direction to draft and revise the text, write the analysis scripts, and launch the experiment runs. The authors designed the study, verified every number against the run artifacts, and take full responsibility for the content. Code and run artifacts, including the pre-registration, will be released on acceptance.

REFERENCES

- [1] T. Wang et al., “MotionBricks: Scalable Real-Time Motions with Modular Latent Generative Model and Smart Primitives,” ACM TOG 45(4), SIGGRAPH 2026, doi:10.1145/3811334, arXiv:2604.24833.
- [2] X. Yang et al., “LAMP: Latent Motion Prior-Guided Real-World Learning for Dexterous Hand Manipulation,” arXiv:2607.06323, 2026.
- [3] H. Lee, H. Jung, S.-H. Heo, S. Yang, “DexSynRefine: Synthesizing and Refining Human-Object Interaction Motion for Physically Feasible Dexterous Robot Actions,” arXiv:2605.05925, 2026.
- [4] C. Berg, V. Caggiano, V. Kumar, “SAR: Generalization of Physiological Agility and Dexterity via Synergistic Action Representation,” RSS 2023, arXiv:2307.03716.

- [5] V. Caggiano, S. Dasari, V. Kumar, “MyoDex: A Generalizable Prior for Dexterous Manipulation,” ICML 2023, arXiv:2309.03130.
- [6] E. Y. Puang, F. Ceola, G. Pasquale, L. Natale, “PCHands: PCA-based Hand Pose Synergy Representation on Manipulators with N-DoF,” IEEE-RAS Humanoids 2025, arXiv:2508.07945.
- [7] H. Yuan, B. Zhou, Y. Fu, Z. Lu, “Cross-Embodiment Dexterous Grasping with Reinforcement Learning,” arXiv:2410.02479, 2024.
- [8] S. Bahl, M. Mukadam, A. Gupta, D. Pathak, “Neural Dynamic Policies for End-to-End Sensorimotor Learning,” NeurIPS 2020, arXiv:2012.02788.
- [9] P. Rivera, E. Valarezo Añazco, T.-S. Kim, “Object Manipulation with an Anthropomorphic Robotic Hand via Deep Reinforcement Learning with a Synergy Space of Natural Hand Poses,” Sensors 21(16):5301, 2021, doi:10.3390/s21165301.
- [10] Z. Luo, J. Cao, S. Christen, A. Winkler, K. Kitani, W. Xu, “Omnigrasp: Grasping Diverse Objects with Simulated Humanoids,” arXiv:2407.11385, 2024.
- [11] S. Linderman, M. Johnson, A. Miller, R. Adams, D. Blei, L. Paninski, “Bayesian Learning and Inference in Recurrent Switching Linear Dynamical Systems,” AISTATS 2017, PMLR 54:914–922.
- [12] Z. Dong, B. A. Seybold, K. Murphy, H. H. Bui, “Collapsed Amortized Variational Inference for Switching Nonlinear Dynamical Systems,” ICML 2020, PMLR 119:2638–2647.
- [13] H. Zhang, S. Starke, T. Komura, J. Saito, “Mode-Adaptive Neural Networks for Quadruped Motion Control,” ACM TOG 37(4), Article 145, SIGGRAPH 2018.
- [14] G. Konidaris, A. Barto, “Skill Discovery in Continuous Reinforcement Learning Domains using Skill Chaining,” NIPS 22, pp. 1015–1023, 2009.
- [15] Y. Lee, J. J. Lim, A. Anandkumar, Y. Zhu, “Adversarial Skill Chaining for Long-Horizon Robot Manipulation via Terminal State Regularization,” CoRL 2021, arXiv:2111.07999.
- [16] Y. Chen, C. Wang, L. Fei-Fei, C. K. Liu, “Sequential Dexterity: Chaining Dexterous Policies for Long-Horizon Manipulation,” CoRL 2023, arXiv:2309.00987.
- [17] O. Nachum, H. Tang, X. Lu, S. Gu, H. Lee, S. Levine, “Why Does Hierarchy (Sometimes) Work So Well in Reinforcement Learning?,” NeurIPS 2019 Deep RL Workshop, arXiv:1909.10618.
- [18] Z. Wen, D. Precup, M. Ibrahimi, A. Barreto, B. Van Roy, S. Singh, “On Efficiency in Hierarchical Reinforcement Learning,” NeurIPS 2020.
- [19] A. Mandlekar et al., “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation,” CoRL 2021, arXiv:2108.03298.
- [20] V. Kurenkov, S. Kolesnikov, “Showing Your Offline Reinforcement Learning Work: Online Evaluation Budget Matters,” ICML 2022, arXiv:2110.04156.
- [21] Y. Liu, H. Yang, X. Si, L. Liu, Z. Li, Y. Zhang, Y. Liu, L. Yi, “TACO: Benchmarking Generalizable Bimanual Tool-ACTION-Object Understanding,” CVPR 2024, pp. 21740–21751, arXiv:2401.08399.
- [22] J. Romero, D. Tzionas, M. J. Black, “Embodied Hands: Modeling and Capturing Hands and Bodies Together,” ACM TOG 36(6), Article 245, SIGGRAPH Asia 2017.
- [23] O. Taheri, N. Ghorbani, M. J. Black, D. Tzionas, “GRAB: A Dataset of Whole-Body Human Grasping of Objects,” ECCV 2020, LNCS 12349, pp. 581–600, arXiv:2008.11200.
- [24] Y.-W. Chao et al., “DexYCB: A Benchmark for Capturing Hand Grasping of Objects,” CVPR 2021, arXiv:2104.04631.
- [25] X. Zhan et al., “OakInk2: A Dataset of Bimanual Hands-Object Manipulation in Complex Task Completion,” CVPR 2024, arXiv:2403.19417.
- [26] I. Higgins et al., “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR 2017.
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [28] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, “MuJoCo Menagerie: A Collection of High-Quality Simulation Models for MuJoCo,” 2022. https://github.com/google-deepmind/mujoco_menagerie
- [29] E. Todorov, T. Erez, Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” IROS 2012, pp. 5026–5033.
- [30] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, “Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations,” RSS 2018, arXiv:1709.10087. Environments as packaged in Gymnasium-Robotics 1.4.2.
- [31] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine, “D4RL: Datasets for Deep Data-Driven Reinforcement Learning,” arXiv:2004.07219, 2020.
- [32] L. Yang et al., “OakInk: A Large-scale Knowledge Repository for Understanding Hand-Object Interaction,” CVPR 2022, arXiv:2203.15709.